Machine Learning

Giancarlo Pozzo

Contents

- Problem: given the data, is a transaction fraudulent?
- Math & Implementation
- Results
- Improving the model
- Other applications & limits

Data

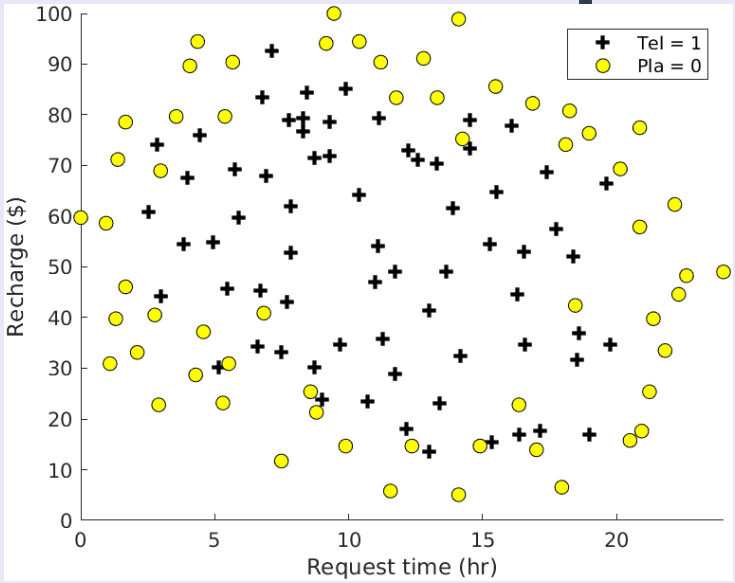
- Recharge amount
- Time of the request

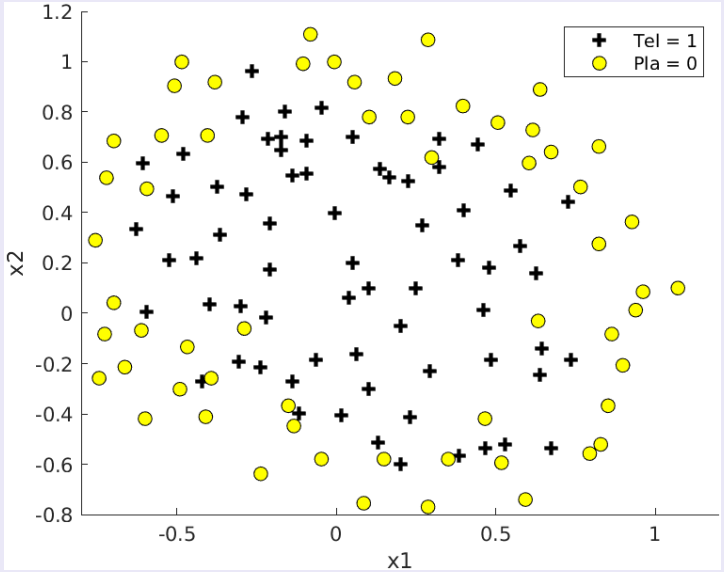
Data

- Recharge amount
- Time of the request
- Location
- Game
- Model of smartphone used
- Other collected data ...
- ...

Data

- **Recharge amount**
- **Time of the request**





	x1	x2	y
	0.051267	0.69956	1
	-0.092742	0.68494	1
	-0.21371	0.69225	1
	-0.375	0.50219	1
	-0.51325	0.46564	1
	-0.52477	0.2098	1
	-0.39804	0.034357	1
X1	-0.30588	-0.19225	1
	0.016705	-0.40424	1
	0.13191	-0.51389	1
	0.38537	-0.56506	1
	0.52938	-0.5212	1
	0.63882	-0.24342	1
	0.73675	-0.18494	1
	⋮	⋮	⋮

X2 Y

More features

Boundary is not linear → add more features

```
degree = 6;
out = ones(size(X1));
for i = 1:degree
    for j = 0:i
        out(:, end+1) = (X1.^(i-j)).*(X2.^j);
    end
end
```

$$\begin{bmatrix} 1 \\ x_1 \\ x_2 \end{bmatrix} \Longrightarrow \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ x_1^2 \\ x_1 x_2 \\ x_2^2 \\ \vdots \\ x_2^6 \end{bmatrix} = x$$

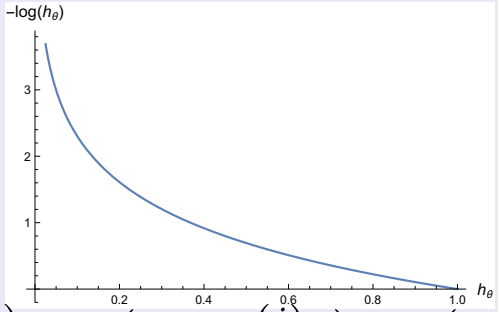
Cost Function

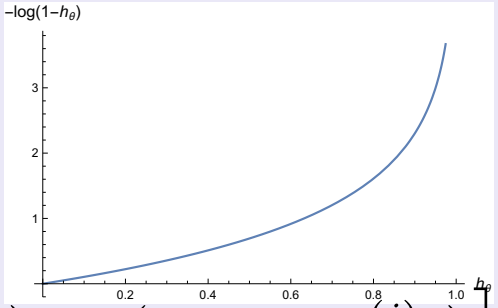
$$J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \left[-y^{(i)} \log\left(h_\theta(x^{(i)})\right) - \left(1 - y^{(i)}\right) \log\left(1 - h_\theta(x^{(i)})\right) \right] + \frac{\lambda}{2m} \sum_{j=1}^{n} \theta_j^2$$

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^{m} \left[\left(h_\theta(x^{(i)}) - y^{(i)} \right) x_j^{(i)} \right] + \frac{\lambda}{m} \theta_j$$

$$h_\theta = g\left(\theta^{\mathrm{T}} x\right) \qquad g(t) = \frac{1}{1 + \mathrm{e}^{-t}}$$

Cost Function

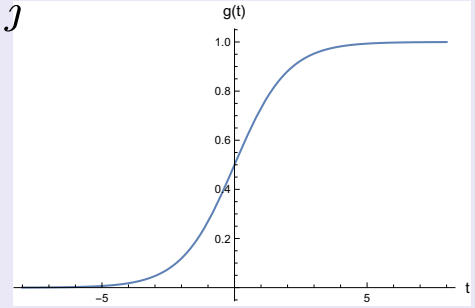




$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}\left[-y^{(i)}\log\left(h_\theta(x^{(i)})\right) - \left(1-y^{(i)}\right)\log\left(1-h_\theta(x^{(i)})\right)\right] + \frac{\lambda}{2m}\sum_{j=1}^{n}\theta_j^2$$

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m}\sum_{i=1}^{m}\left[\left(h_\theta(x^{(i)}) - y^{(i)}\right)x_j^{(i)}\right] + \frac{\lambda}{m}\theta_j$$

$$h_\theta = g\left(\theta^{\mathrm{T}}x\right) \qquad g(t) = \frac{1}{1+\mathrm{e}^{-t}}$$



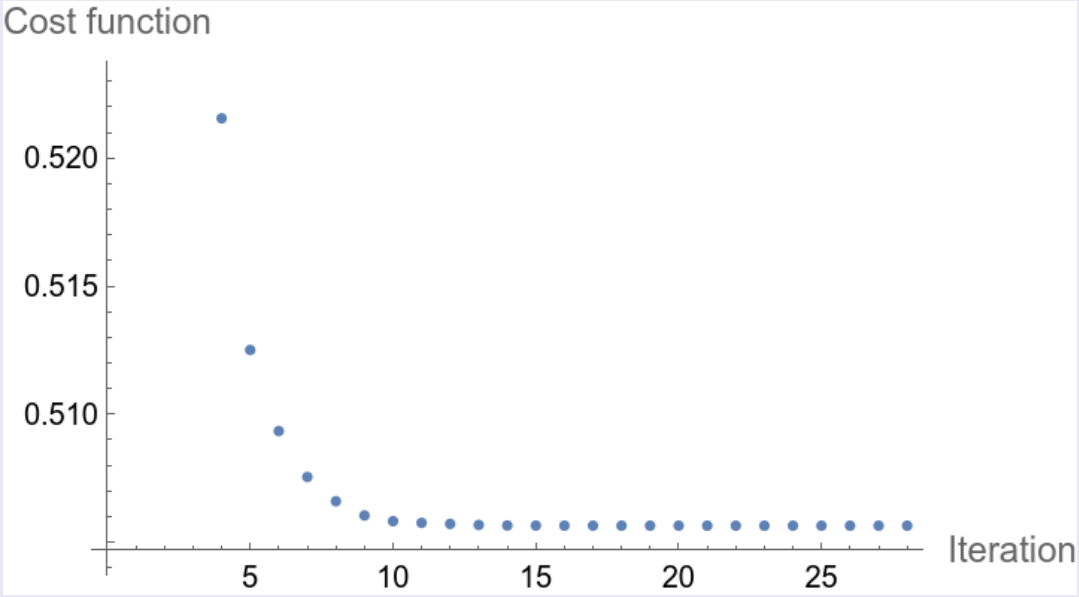
Iterations

Iteration	f(x)	Step-size	First-order optimality
0	0.693147		0.0435
1	0.615078	10	0.058
2	0.561446	1	0.0931
3	0.521555	1	0.0204
4	0.512506	1	0.00962
5	0.509341	1	0.00871
6	0.507547	1	0.0045
7	0.506598	1	0.00478
8	0.506042	1	0.00325
9	0.505819	1	0.0019
10	0.505755	1	0.000831
11	0.505713	1	0.000955
12	0.505672	1	0.00111
13	0.505652	1	0.000564
14	0.505648	1	0.000172
15	0.505646	1	0.000144
16	0.505645	1	0.000164
17	0.505644	1	0.000125
18	0.505644	1	6.2e-05
19	0.505644	1	5.85e-05
20	0.505644	1	5.54e-05
21	0.505643	1	2.73e-05
22	0.505643	1	8.26e-06
23	0.505643	1	3.17e-06
24	0.505643	1	3e-06
25	0.505643	1	2.42e-06
26	0.505643	1	2.35e-06
27	0.505643	1	1.01e-06

Local minimum found.

Optimization completed because the size of the gradient is less than the value of the optimality tolerance.

Iterations



13	0.505652	1	0.000564
14	0.505648	1	0.000172
15	0.505646	1	0.000144
16	0.505645	1	0.000164

Iteration	f(x)	Step-size	First-order optimality
17	0.505644	1	0.000125
18	0.505644	1	6.2e-05
19	0.505644	1	5.85e-05
20	0.505644	1	5.54e-05
21	0.505643	1	2.73e-05
22	0.505643	1	8.26e-06
23	0.505643	1	3.17e-06
24	0.505643	1	3e-06
25	0.505643	1	2.42e-06
26	0.505643	1	2.35e-06
27	0.505643	1	1.01e-06

Local minimum found.

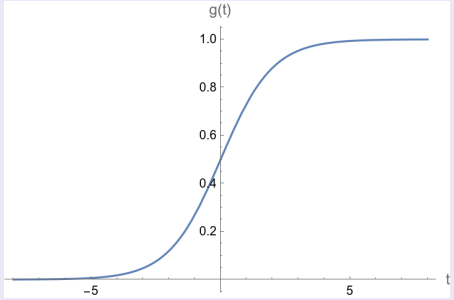
Optimization completed because the size of the gradient is less than the value of the optimality tolerance.

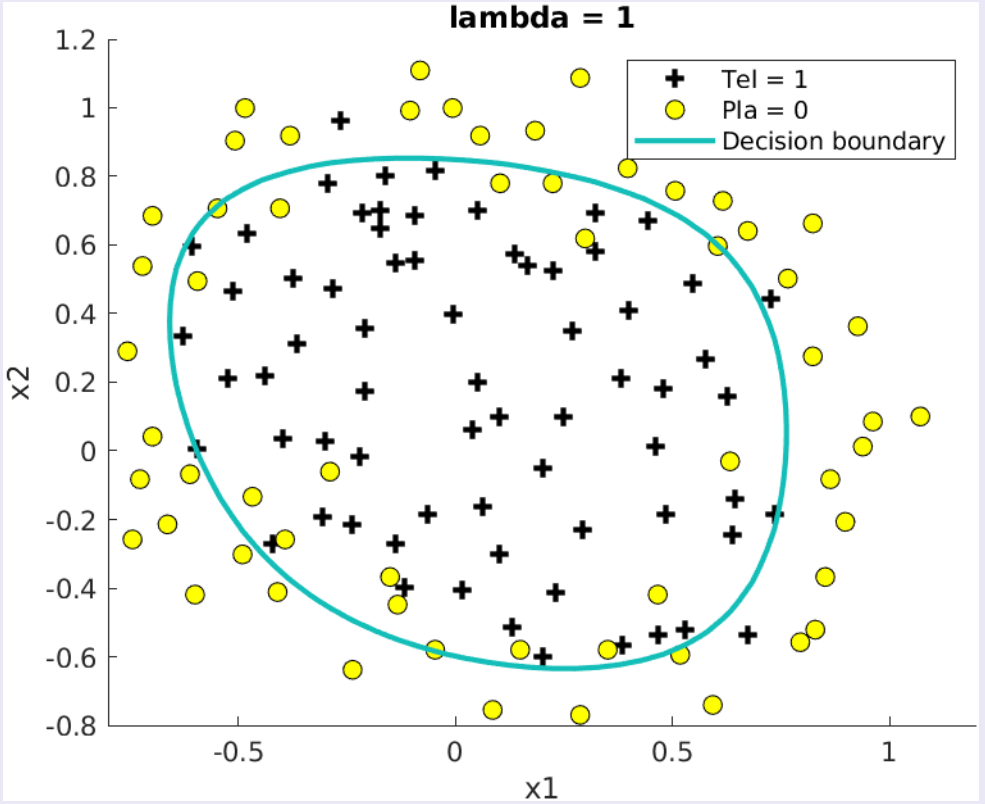
Example, decision boundary & accuracy

Time: 00:00
Recharge $100

$$\begin{bmatrix} 1 \\ x_1 \\ x_2 \end{bmatrix} \equiv \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \Longrightarrow \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix} = x$$

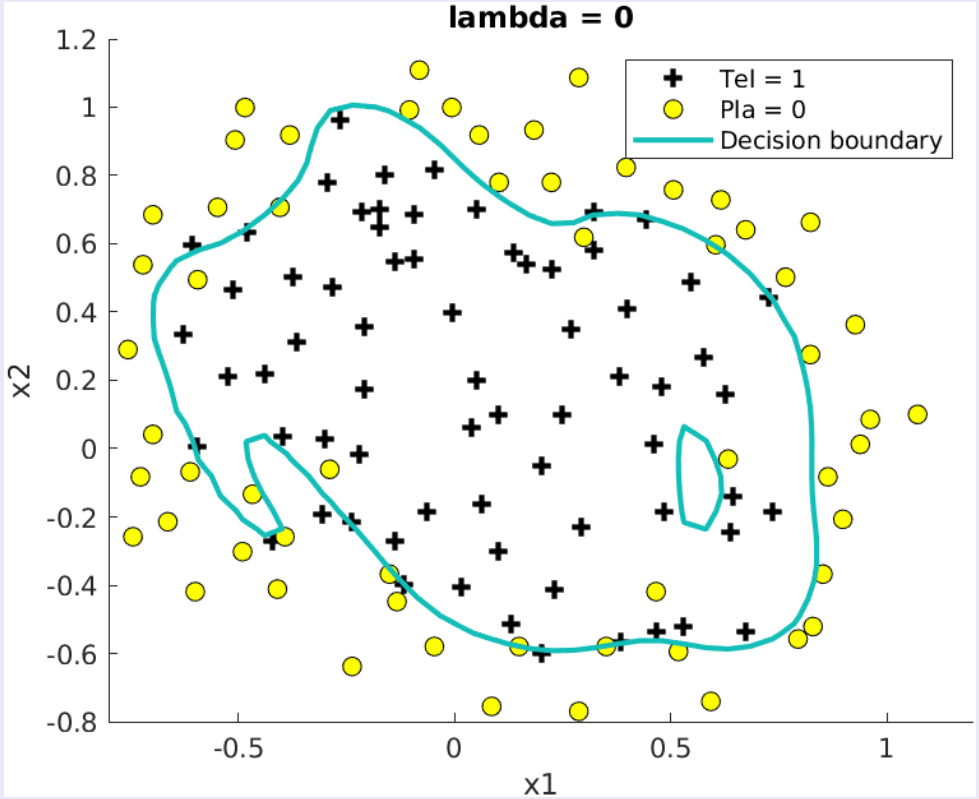
$$x\,\theta = -10.39 < 0$$

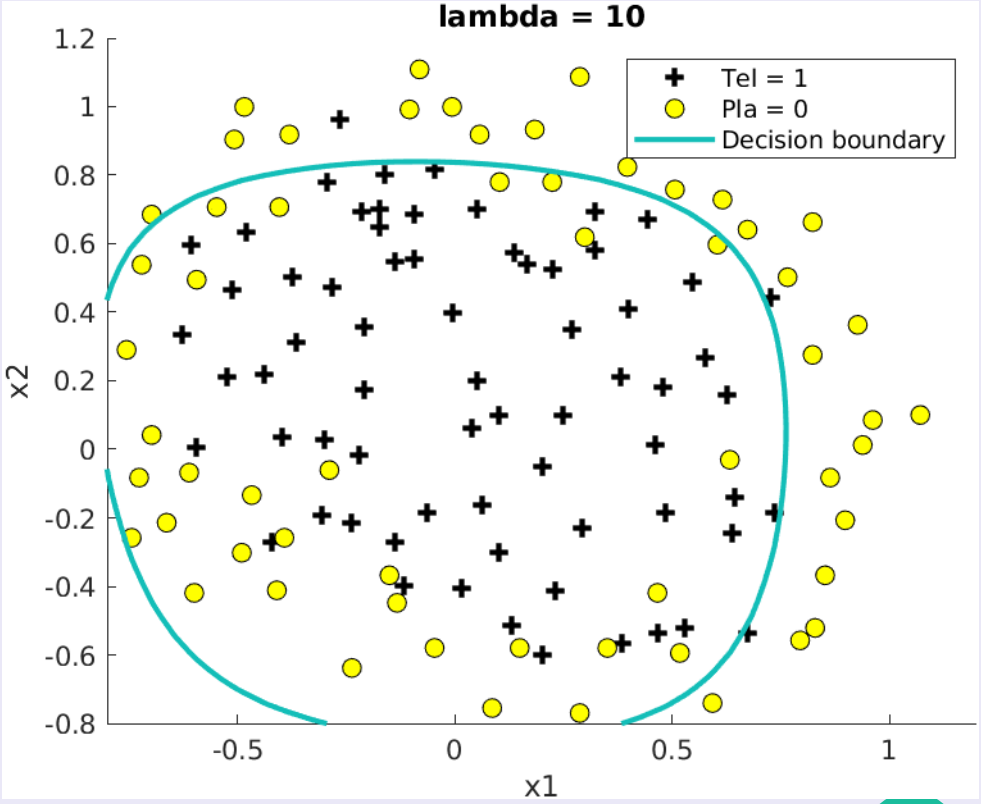




Train accuracy
84.4%

Different lambdas





Model selection

Training 60%, Cross validation 20%, Test 20%

```
lambdaVsJcv = [];
for lambda = 0:0.0001:0.03
```

$$\min_{\theta} J(X_{\text{tr}}, Y_{\text{tr}}, \theta) = \min_{\theta} \sum_{i=1}^{m} \left[-y_{\text{tr}}^{(i)} \log\left(h_\theta(x_{\text{tr}}^{(i)})\right) - \left(1 - y_{\text{tr}}^{(i)}\right) \log\left(1 - h_\theta(x_{\text{tr}}^{(i)})\right) \right] + \frac{\lambda}{2} \sum_{j=1}^{n} \theta_j^2$$

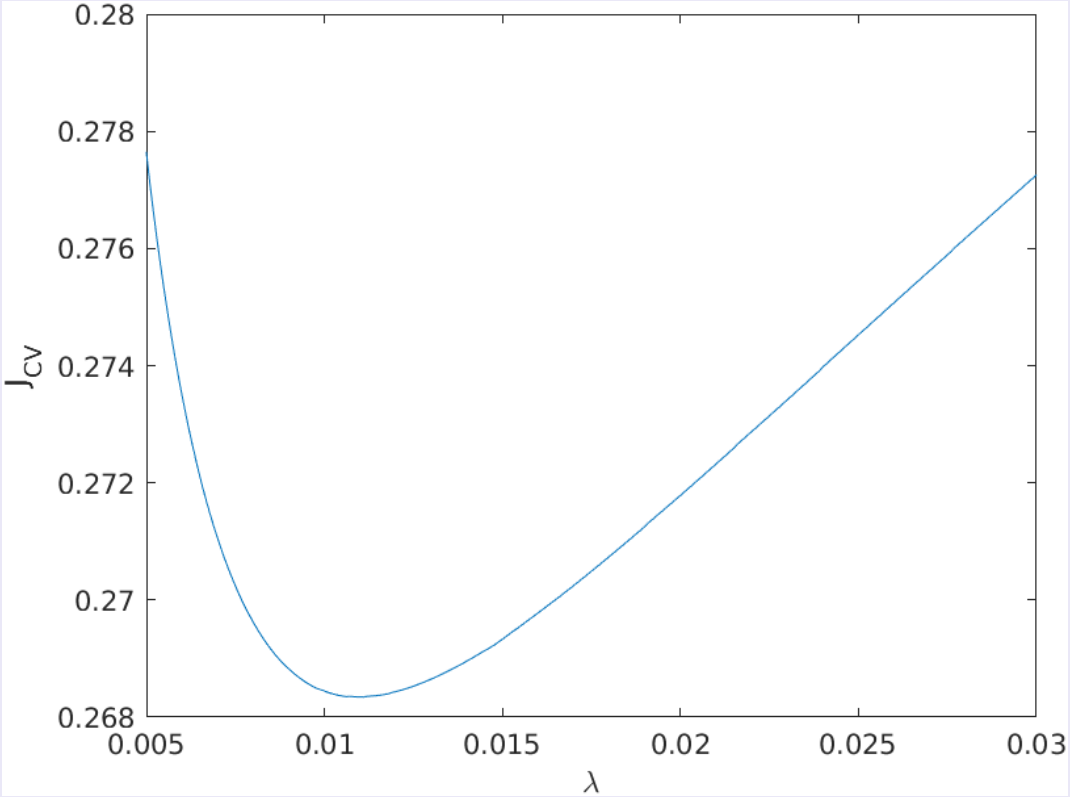
$$\longrightarrow \theta_\lambda \longrightarrow J_{\text{cv}}(X_{\text{cv}}, Y_{\text{cv}}, \theta_{\text{tr}}) = \sum_{i=1}^{m} \left[-y_{\text{cv}}^{(i)} \log\left(h_\theta(x_{\text{cv}}^{(i)})\right) - \left(1 - y_{\text{cv}}^{(i)}\right) \log\left(1 - h_\theta(x_{\text{cv}}^{(i)})\right) \right]$$

```
    lambdaVsJcv = [lambdaVsJcv; lambda,    ];
```

J_{cv}

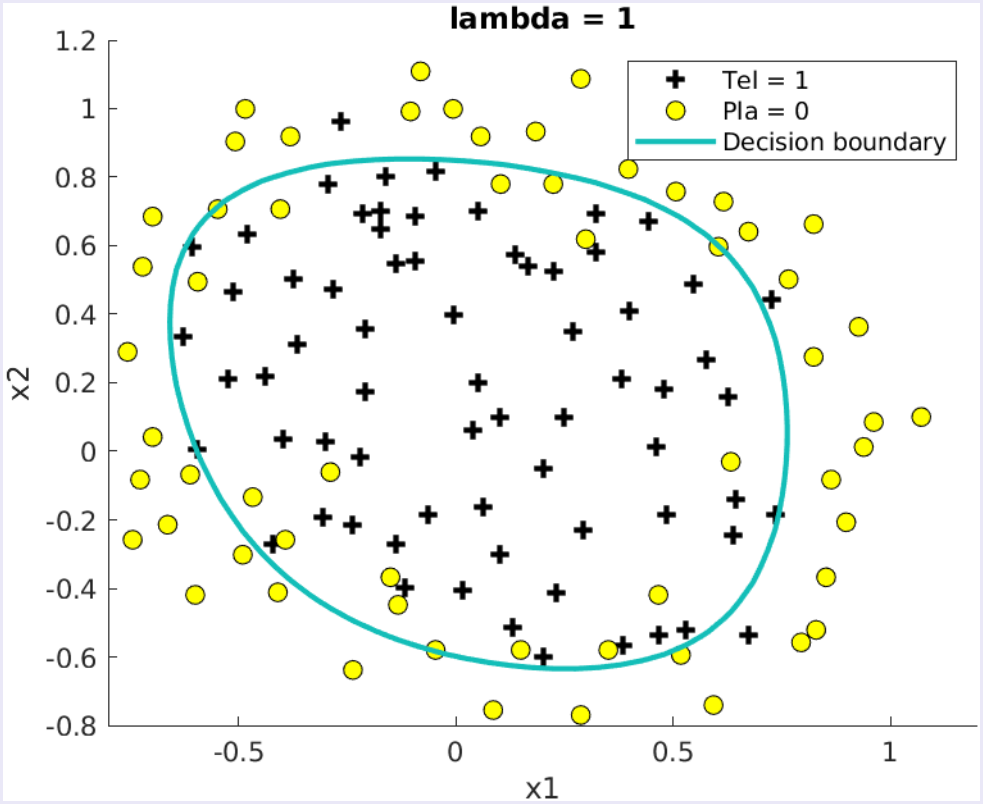
```
end
```

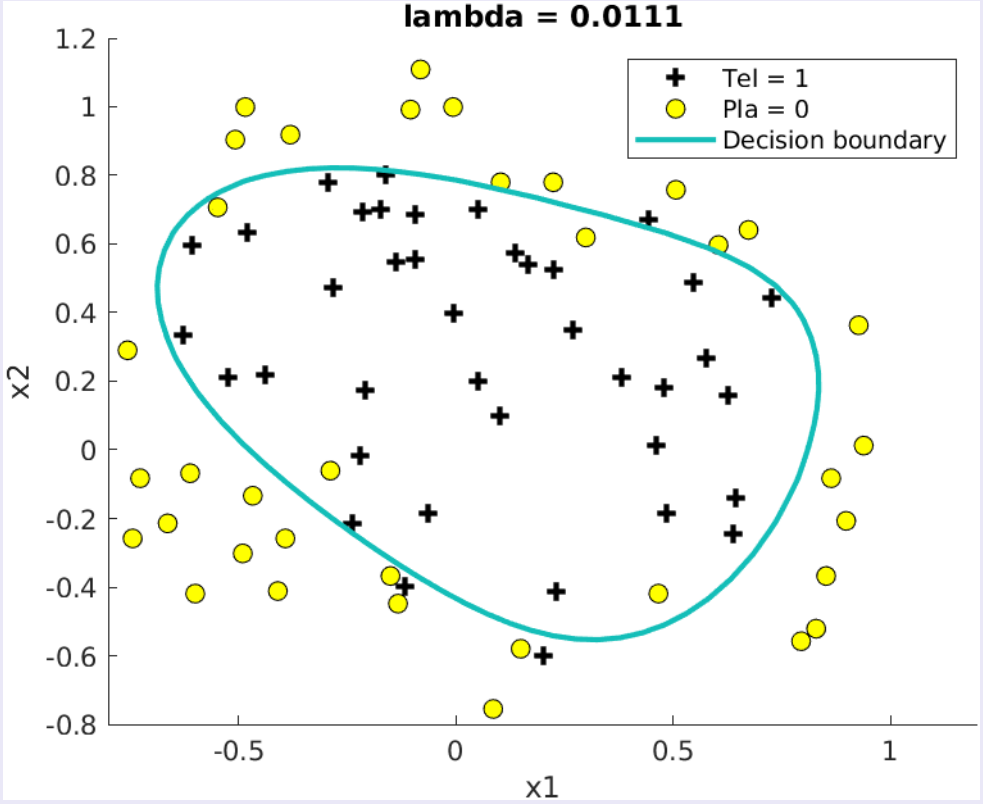
Cross validation: Lambda Vs J_cv



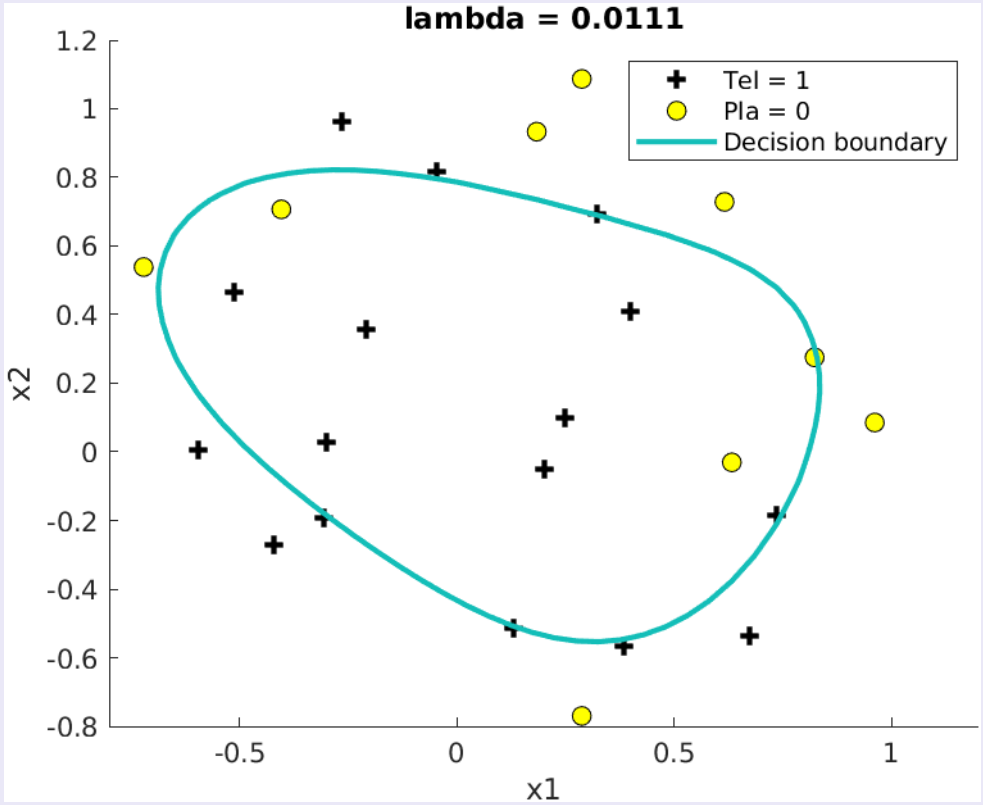
$\lambda = 0.0111$ minimise J_{cv}

Best lambda


lambda = 1
Tel = 1
Pla = 0
Decision boundary
x1
x2
1.2
1
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
-0.5
0
0.5
1


lambda = 0.0111
Tel = 1
Pla = 0
Decision boundary
x1
x2
1.2
1
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
-0.5
0
0.5
1

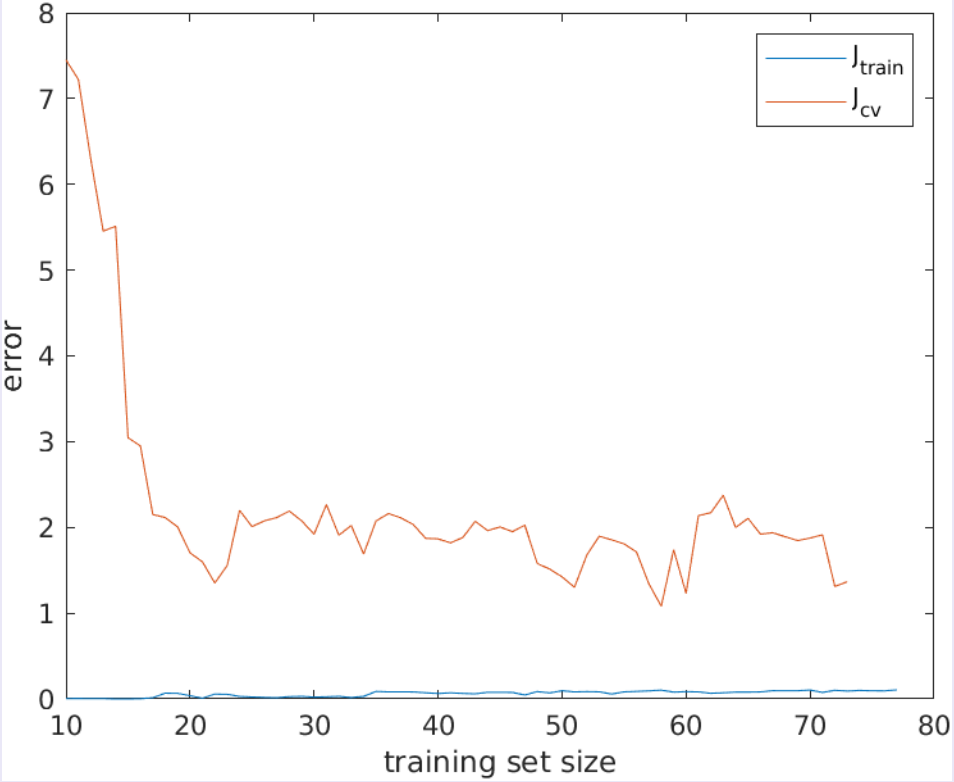
Test: hypothesis evaluation

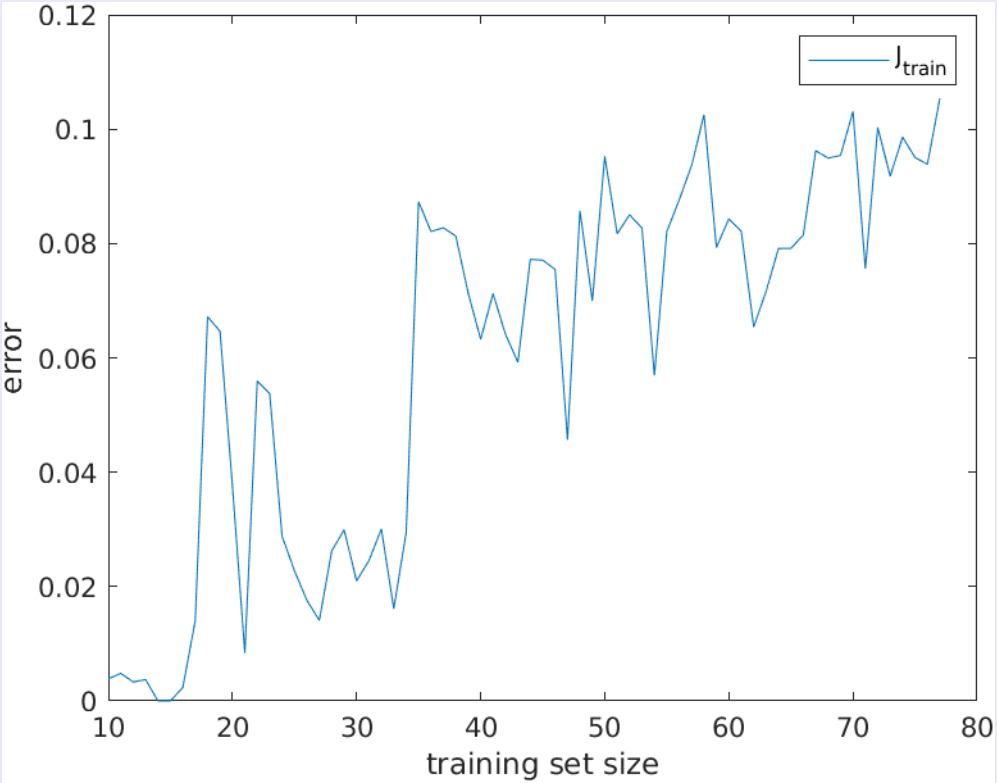


Only 52% of test data is correctly predicted.

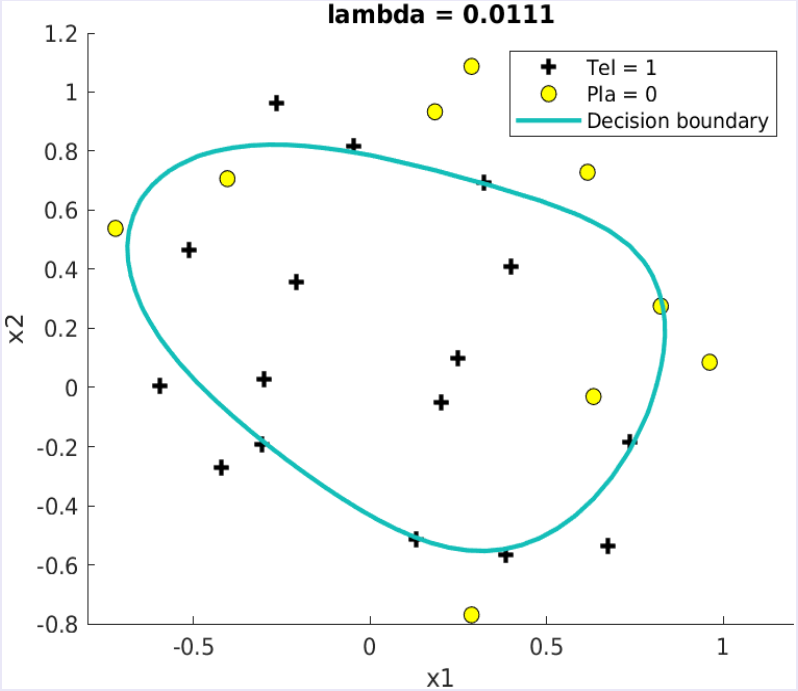
How can we improve this?

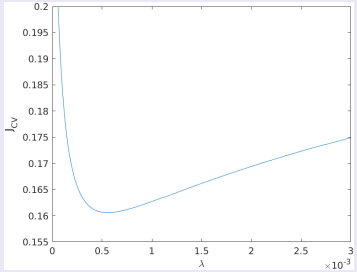
More data

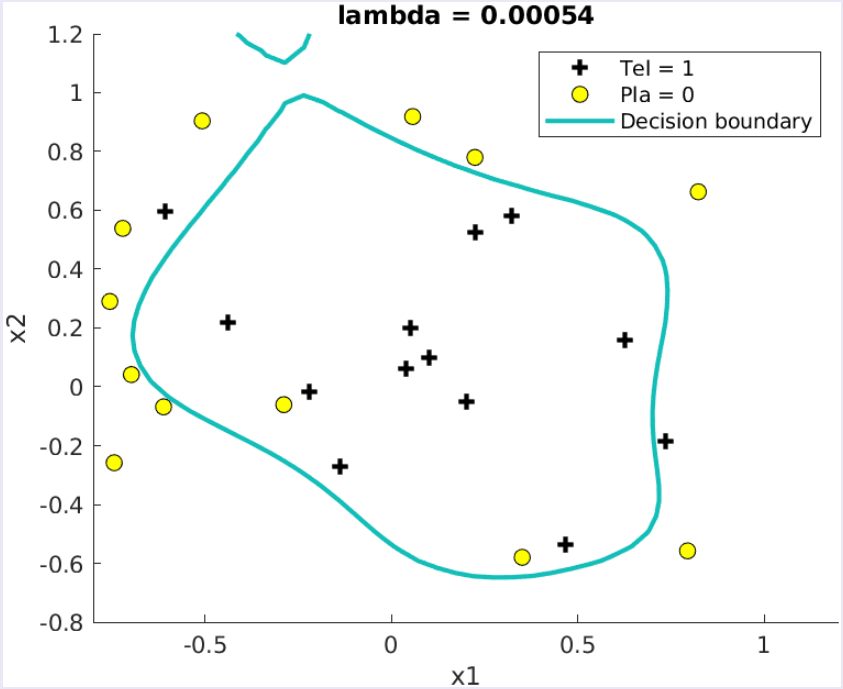

J_{train}
J_{cv}
error
training set size
0
1
2
3
4
5
6
7
8
10
20
30
40
50
60
70
80


J_{train}
error
training set size
0
0.02
0.04
0.06
0.08
0.1
0.12
10
20
30
40
50
60
70
80

Data shuffle







52% of test data is correctly predicted

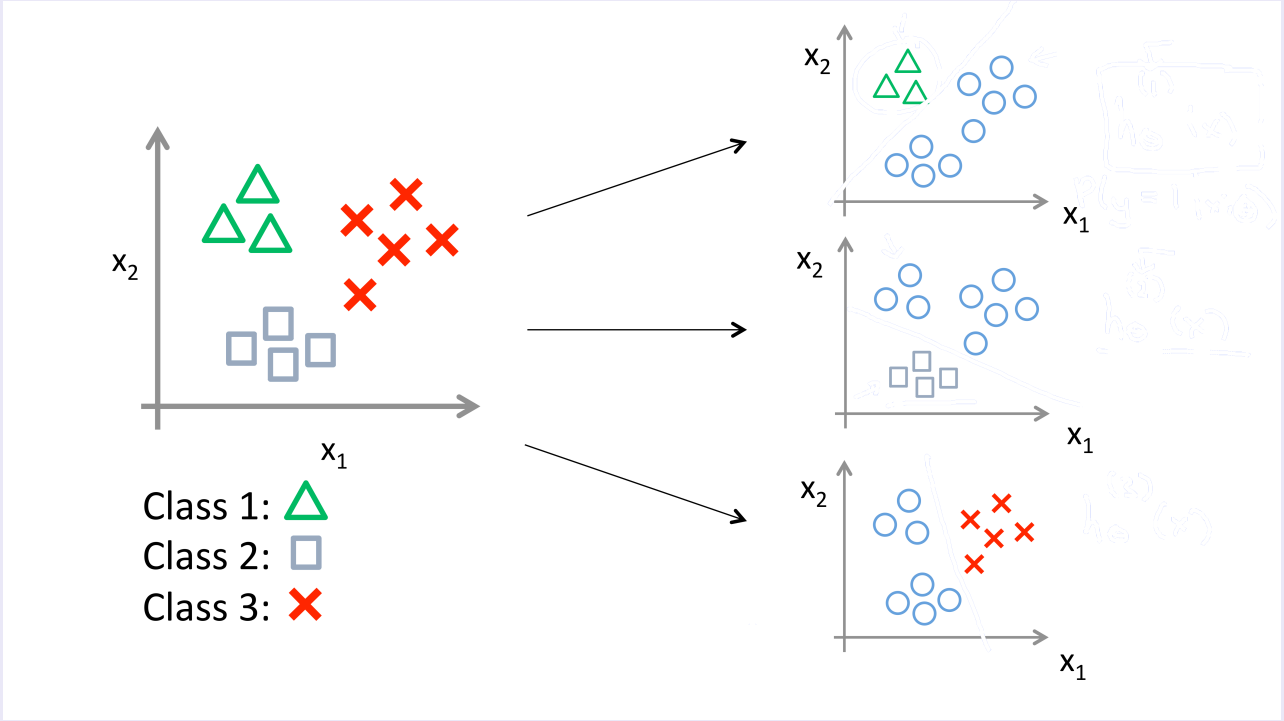
84% of test data is correctly predicted

Multi-class classification

One vs All

$$\theta \in \mathbb{R}^{K(N+1)}$$

$$\max_K h_\theta^K(x)$$



Other applications

- Online transactions: fraudulent or not?
- Emails/text: spam or not
- Pre-filtering: urgent or not-urgent ticket
- Medical: tumour or not
- Text recognition: 0,1,2,...,9
- ...

Limits

- Limit case: predict machine failure, PCA to reduce dimensions and use logistic regression
- Computer vision, NN
- Cybersecurity: malware detection (binary executables) random forest or deep-learning on byte/sequence features
- Natural language moderation (toxic vs. allowed contents) Transformer (BERT) fine-tuned for binary toxicity

Thank you